ЦИФРОВОЙ СПАЙКОВЫЙ ОРГАНИЗМ · ВЕРСИЯ 4.0

NEUROWEAVE / SPIKER

Единый архитектурный концепт: философия, топология, нейрохимия, трёхфазное обучение и реализация Этапа 0. Канонический синтез манифеста idea.md (v3.0), спецификации start.md (v3.1) и кода фундаментального ядра.

Исследовательский проект NEUROWEAVE

Статус: Proof of Concept с перспективой масштабирования

Спайковые нейронные сети · нейромодуляция · непрерывное обучение

Октябрь 2026

Содержание

Аннотация и статус документа 3

1 Философия и парадигма 4

2 Макроархитектура: топология организма 5

3 Микроархитектура: мульти-компартментный ALIF-нейрон 7

4 Нейрохимическая система: два контура регуляции 9

5 Синапс с состоянием: две шкалы памяти и следы пригодности 10

6 SON: нейроны само-наблюдения и межзональная шина 12

7 Трёхфазная методология обучения 12

8 Вычислительная архитектура: Dual-Channel Engine 14

9 Реализация Этапа 0: фундаментальное ядро 15

10 Дорожная карта и файловая структура проекта 25

11 Ожидаемые свойства системы 27

12 Аналитические приложения 28

13 Глоссарий терминов и сокращений 30

14 Заключение: следующий шаг — Этап 1 32

Аннотация и статус документа

Настоящий документ представляет собой единую каноническую сводку архитектурного концепта **NEUROWEAVE / SPIKER** — цифрового спайкового организма, в котором вычисление, память и обучение объединены в один непрерывный процесс, протекающий во времени. Документ синтезирует три исходных материала проекта в одну непротиворечивую спецификацию: философско-архитектурный манифест *«Цифровой Спайковый Организм»* (idea.md, v3.0), техническую спецификацию и дорожную карту *start.md* (v3.1), а также реализацию Этапа 0 — четырёх базовых модулей вычислительного ядра с верификационными тестами. Ничего из исходных материалов не выброшено: все механизмы, формулы, решения дорожной карты и код вошли в настоящий документ; расхождения между версиями устранены путём принятия уточнённых формулировок в качестве канона.

Принцип синтеза. Версия v3.0 задавала философию и состав подсистем, v3.1 — их математическую формализацию. Однако между ними оставалось три математических расхождения, выявленных на этапе перехода к коду: интеграция ацетилхолина (ACh) в уравнение сомы, честная $O(N)$-факторизация следов пригодности и истинная $O(1)$-память в Фазе 1. В настоящей сводке (v4.0) эти три уточнения *интегрированы в основной текст как канонические*: именно в уточнённом виде формулы реализованы в коде Этапа 0 и должны считаться действующими. Краткая сводка отличий приведена в таблице 1.

Таблица 1. Что нового в канонической версии v4.0 относительно v3.0/v3.1.

Уточнение	**Суть канонической формулировки**
ACh в уравнении сомы	Ацетилхолин введён в ток сомы как динамический модулятор проводимости: $(1 + \gamma_{\text{ACh}}\,\text{ACh})$ усиливает базальный AMPA-ток и симметрично $(1 - \gamma_{\text{ACh}}\,\text{ACh})$ ослабляет апикальный NMDA-ток (разд. 3).
Честная память $O(N)$ следов	Накапливаемая во времени матрица следов пригодности $\mathcal{E}_{ij}$ перестаёт иметь ранг 1 и требует $O(N^2)$ памяти. Вместо неё хранятся два *вектора* следов на уровне нейронов, а обновление w_{fast} выполняется внешним произведением векторов в момент прихода дофамина (разд. 5).
$O(1)$-память Фазы 1	Для исключения построения BPTT-графа вглубь времени на каждом такте выполняется `detach()` мембранного потенциала $V(t-1)$ и адаптации $a(t-1)$; суррогатный градиент проходит строго через текущие синаптические токи (разд. 7.1).

Статус и роль документа. NEUROWEAVE — исследовательский проект уровня Proof of Concept с перспективой масштабирования. Настоящий документ выполняет роль единственного источника правды (*single source of truth*) для инженерной команды: он одновременно фиксирует парадигму, макро- и микроархитектуру, нейрохимию, методологию обучения, вычислительную модель, готовый код фундаментального ядра (Этап 0) и детализированную дорожную карту Этапов 1–5. Документ адресован разработчикам системы и техническим рецензентам; от него следует отталкиваться при написании любых новых модулей проекта `Spiker/`.

> **Суть проекта в одной фразе.** NEUROWEAVE — это цифровое воспроизведение ключевых *биологических механизмов* мозга (компартментность, нейрохимия, сон, конкуренция), а не симуляция его внешнего поведения и не очередная обёртка над плотными матрицами.

1 Философия и парадигма

1.1 Предел статических аппроксиматоров

Современные большие языковые модели — это статические функции аппроксимации $f(x) = y$: после завершения обучения веса заморожены, и любой прогон инференса не меняет систему ни на бит. Такое разделение на «фазу обучения» и «фазу инференса» имеет три неудобных следствия. Во-первых, модель физически не способна адаптироваться к новому опыту в момент работы: любое обновление требует отдельного цикла дообучения со всеми сопутствующими рисками. Во-вторых, дообучение на новых данных почти неизбежно разрушает старые навыки — феномен катастрофического забывания, с которым приходится бороться внешними по отношению к модели средствами. В-третьих, вся «память» системы живёт только в весах: между сессиями модель не накапливает ни состояния, ни опыта, ни истории собственной активности.

NEUROWEAVE предлагает другую онтологию: *непрерывный адаптивный субстрат*. Цель — создать систему, где вычисление, память и обучение являются единым процессом, протекающим во времени. В такой системе нет отдельно «фазы инференса» и «фазы обучения»: сеть учится в момент генерации, меняя свою «температуру» и «внимание» через внутренние нейромодуляторы. Память существует в нескольких временах — от миллисекундных следов пригодности до долговременных кристаллизованных весов, — и переходы между этими шкалами управляются самой системой, а не внешним расписанием.

1.2 Отличие от SNN-фреймворков

Столь же принципиально размежевание с существующими спайковыми фреймворками. Типичный подход индустрии — использовать спайки как *декорацию* для плотных матриц: событийный фасад, за которым скрывается обычный тензорный движок с глобальным обратным распространением. В NEUROWEAVE спайки, задержки и локальная пластичность — это *фундамент* вычислительного графа, а не оформление. Из этого следует конкретная инженерная дисциплина: каждая подсистема должна либо использовать спайковую событийность по существу (гейтинг, экономия вычислений), либо честно обосновывать, почему в данном узле выбраны плотные тензоры (как в случае быстрого контура GPU, см. разд. 8).

> **Парадигмальная формула проекта:** вычисление = память = обучение. Это один процесс, развёрнутый во времени, а не три подсистемы, синхронизируемые извне. Всё остальное — топология, нейрохимия, сон — следствия этой формулы.

1.3 Три столпа концепта

Синтез исходных материалов позволяет выделить три несущие конструкции, на которых держится вся архитектура. Первая — *асимметрия как ресурс*: два полушария с намеренно разными временными константами, скоростями обучения и уровнем внутреннего шума не дублируют, а дополняют друг друга (разд. 2). Вторая — *встроенная нейрохимия*: дофамин, серотонин, ацетилхолин и норадреналин реализованы не глобальными скалярами, а векторными тензорами, привязанными к топологии зон и управляющими пластичностью на уровне отдельных анклавов (разд. 4). Третья — *жизненный цикл*: организм не только бодрствует и учится, но и *спит* — консолидирует быстрые веса в медленные, чистит шум и рекалибрует буферы (разд. 7.3). Вместе эти столпы дают то, что мы называем цифровым организмом: систему, которая учится, спит, конкурирует внутри себя и адаптируется.

2 Макроархитектура: топология организма

Система строится снизу вверх, избегая глобального «монолита»: сначала автономные функциональные единицы, затем — ограниченные и управляемые каналы их взаимодействия, и лишь в конце — механизм коллективного решения. Такой порядок гарантирует, что каждая часть организма полезна сама по себе и что сборка никогда не подменяет локальную специализацию глобальной оптимизацией. Общая топология показана на рис. 1.

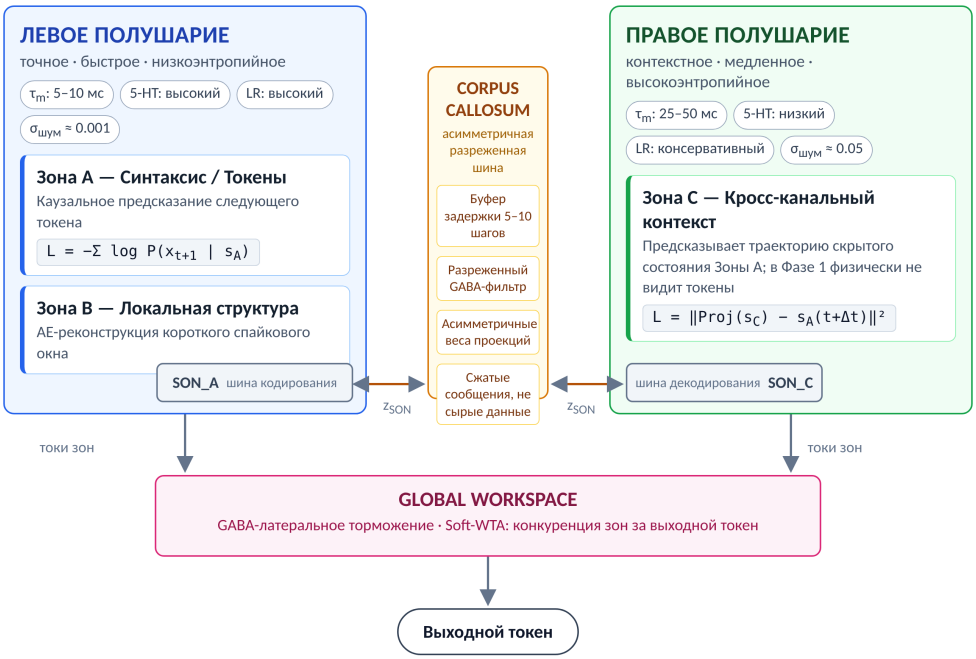


Рис. 1. Макро-топология организма: асимметричные полушария с изолированными зонами, шина Corpus Callosum, конкуренция в Global Workspace.

2.1 Функциональные зоны: локальные рекуррентные анклавы

Базовая строительная единица системы — *зона*: замкнутый рекуррентный анклав нейронов (например, «Синтаксис», «Контекст», «Локальная структура»), который обладает тремя определяющими свойствами. Во-первых, у каждой зоны есть *собственная локальная цель обучения* — свой лосс, никак не зависящий от глобального обратного распространения. Во-вторых, зона рекуррентна: её состояние в момент t зависит от её же истории, что даёт естественную короткую память без внешних механизмов. В-третьих, зона инкапсулирует свои ресурсы: пул возбуждающих нейронов, пул GABA-интернейронов и собственный ансамбль нейронов само-наблюдения SON (разд. 6). Изоляция зон — не декоративная, а рабочая: в Фазе 1 обучения межзонные связи физически разорваны (маскированы нулями), и каждая зона обязана научиться своей задаче в полном одиночестве (разд. 7.1).

2.2 Полушария и слом симметрии

Мозг функционально эффективен именно благодаря жёсткому нарушению симметрии между полушариями. Если бы две копии одной архитектуры обучались на одних данных с одинаковыми параметрами, они неизбежно выродились бы в дубликаты репрезентаций — две зоны, делающие одну и ту же работу и удваивающие расход памяти и вычислений. Поэтому в NEUROWEAVE

полушария *намеренно асимметричны по своей природе*: они различаются инициализацией, временными константами и всем образом работы (табл. 2).

Таблица 2. Параметры дивергенции полушарий (слом симметрии).

Параметр	Левое (аналитическое)	Правое (ассоциативное)
Роль	точное, быстрое, низкоэнтропийное	контекстное, медленное, высокоэнтропийное
Постоянная мембраны τ_m	короткая: 5–10 мс	длинная: 25–50 мс
База серотонина 5-HT	высокий (детерминизм, жёсткий отбор гипотез)	пониженный (широкий охват паттернов)
Скорость локальной адаптации	высокая (lr высокий)	умеренно-консервативная
Шум мембран σ_{noise}	≈ 0.001 (почти детерминирован)	≈ 0.05 (выраженная стохастика)

Слом симметрии закреплён не только в гиперпараметрах, но и в самих задачах зон (табл. 3). Ключевой приём: Зона C правого полушария в Фазе 1 *физически не видит токены*. Вместо этого она обучается предсказывать траекторию скрытого состояния Зоны A на горизонте Δt:

$$\mathcal{L}_{\text{Zone_C}} = \left\| \text{Proj}\big(s_{\text{Zone_C}}(t)\big) - s_{\text{Zone_A}}(t + \Delta t) \right\|_2^2. \tag{1}$$

Это решение гарантирует, что полушария не выродятся в дубликаты: правое полушарие строит *модель фазового пространства* левого, а не копирует его сенсорную функцию. Зона A, в свою очередь, решает классическую каузальную авторегрессионную задачу:

$$\mathcal{L}_{\text{Zone_A}} = -\sum_{t} \log P\big(x_{t+1} \mid s_{\text{Zone_A}}(\leq t)\big), \tag{2}$$

а Зона B обучается реконструкции короткого спайкового окна — аутоэнкодерскому сжатию локального контекста, которое даёт левой системе компактную внутреннюю репрезентацию её ближайшей истории.

Таблица 3. Специализация зон и их локальные цели в Фазе 1.

Зона	Полушарие	Локальная цель обучения
A — Синтаксис / Токены	левое	каузальное предсказание следующего токена, ур. (2)
B — Локальная структура	левое	реконструкция короткого спайкового окна (аутоэнкодирование)
C — Кросс-канальный контекст	правое	предсказание динамики скрытого состояния Зоны A, ур. (1); токены не видит

2.3 Corpus Callosum: шина сжатых сообщений

Межполушарная связь — это не плотная полностью связная матрица, а *асимметричная разреженная шина* с тремя физиологическими свойствами. Первое — *задержка*: кольцевой буфер на 5–10 шагов, который разносит во времени источник и приёмник сигнала и делает межполушарное взаимодействие принципиально асинхронным. Второе — *торможение*: проходя через шину, сигнал фильтруется разреженным GABA-контуром, так что слабые фоновые пересылки подавляются, а приоритетные пакеты проходят. Третье — *асимметрия весов*: проекции «левое → правое» и «правое → левое» имеют разные обучаемые матрицы, отражающие различие ролей полушарий. Функционально Callosum передаёт *не сами данные, а сжатые сообщения* о них: латентные векторы, которые готовят ансамбли SON (разд. 6). Полоса пропускания шины — намеренно узкое место системы: организм вынужден развивать экономный внутренний язык вместо сырой трансляции состояний.

2.4 Global Workspace: конкуренция за выходной токен

Выход организма не является суммой мнений всех зон — это результат *конкуренции*. Механизм Global Workspace реализует схему Winner-Takes-All / Soft-Attention: зоны «борются» за право влиять на выходной токен, а побеждает доминирующий в данный момент ансамбль. Физическая основа конкуренции — латеральное GABA-торможение: активация доминантной популяции возбуждает тормозной пул, который пропорционально подавляет мембранные потенциалы альтернативных путей (формула (9) в разд. 4). Это создаёт эффект единого *фокуса внимания* и одновременно защищает систему от эпилептиформного спайкового шторма. Без такого торможения одновременная трансляция нескольких несогласованных гипотез на выход порождала бы «шизофренический» текст — хаотичную смесь синтаксических конструкций; GABA-пул гарантирует один когерентный фокус без размытия энтропии.

3 Микроархитектура: мульти-компартментный ALIF-нейрон

Мы отказываемся от простых точечных LIF-нейронов: базовой вычислительной единицей NEUROWEAVE является *трёхсегментный компартментный элемент*, в котором раздельно адресуются базальный дендрит, апикальный дендрит и сома (рис. 2). Компартментность здесь — не усложнение ради биомиметики: она разделяет два потока информации — сенсорный вход снизу и контекстный прогноз сверху — и задаёт строгие правила их взаимодействия вплоть до уровня одного нейрона.

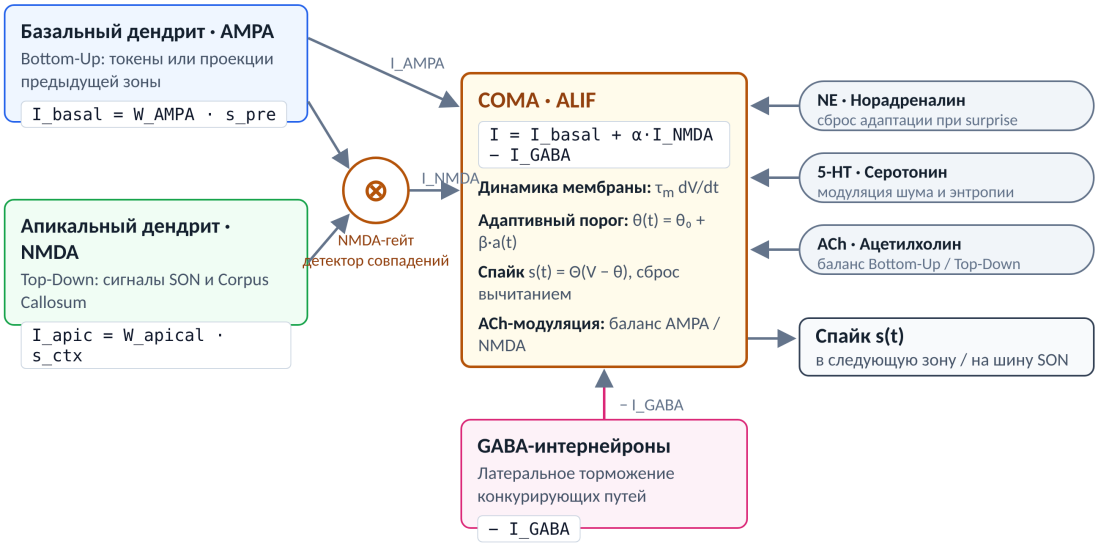


Рис. 2. Мульти-компартментный ALIF-нейрон: базальный (AMPA) и апикальный (NMDA) дендриты, NMDA-детектор совпадений, сома с адаптивным порогом, тормозной GABA-вход и модуляторное управление.

3.1 Базальный дендрит: AMPA-рецепция (Bottom-Up)

Базальный дендрит принимает прямой сенсорный поток — входные токены или проекции от предыдущей локальной зоны. Метаболизм этого пути — стандартные быстрые глутаматные проводимости AMPA-типа:

$$I_{\text{basal}}(t) = W_{\text{AMPA}} \cdot s_{\text{pre}}(t), \tag{3}$$

где $s_{\text{pre}}(t)$ — спайковая маска пресинаптических нейронов, а W_{AMPA} — обучаемая матрица весов. Кинетика затухания тока быстрая ($\tau_{\text{AMPA}} \approx$ 2–5 мс), поэтому базальный путь передаёт именно мгновенную, а не накопленную картину входа: нейрон «видит» конкретный символ здесь и сейчас.

3.2 Апикальный дендрит: NMDA-коинциденция (Top-Down)

Апикальный дендрит принимает модулирующие сигналы сверху: латентные прогнозы от локального ансамбля SON и отложенные проекции Corpus Callosum. В биологии NMDA-каналы заблокированы ионами магния Mg^{2+} при потенциале покоя и открываются *только при одновременном* выполнении двух условий: локальной деполяризации мембраны и связывании глутамата. В NEUROWEAVE это формализуется как нелинейный поэлементный гейтинг:

$$I_{\text{NMDA}}(t) = \sigma\big(I_{\text{basal}}(t) - \theta_{\text{Mg}}\big) \odot \big(W_{\text{apical}} \cdot s_{\text{context}}(t)\big), \tag{4}$$

где σ — калиброванная сигмоида снятия магниевого блока, θ_{Mg} — порог деполяризации, $\odot$ — произведение Адамара, а s_{context} — спайки контекстных источников (SON, Callosum). Это и есть физическая основа внимания на уровне одного нейрона.

> **Семантика NMDA-гейта:** «Вижу входной символ (базальный ток) И контекст предсказывает этот класс (апикальный ток) → взрывной спайк». Обратное неверно: сколько бы сильным ни был абстрактный контекст сверху, без базального входа магниевый блок не снимается — нейрон молчит.

3.3 Ацетилхолиновая модуляция сомы (канон v4.0)

Ацетилхолин балансирует вклад Bottom-Up и Top-Down путей. В исходной спецификации v3.1 он был описан словесно, но не входил в уравнение сомы; в канонической версии v4.0 ACh введён в ток сомы как динамический модулятор проводимости обоих дендритных путей:

$$I_{\text{soma}}(t) = \underbrace{\big(1 + \gamma_{\text{ACh}}\,\text{ACh}\big)}_{\text{усиление входа}} I_{\text{basal}}(t) + \alpha_{\text{NMDA}} \underbrace{\big(1 - \gamma_{\text{ACh}}\,\text{ACh}\big)}_{\text{ослабление контекста}} I_{\text{NMDA}}(t) - I_{\text{GABA}}(t), \tag{5}$$

где $\gamma_{\text{ACh}} \in [0, 1]$ — коэффициент модуляции, а уровень $\text{ACh} \in [-1, 1]$ — компонента векторного тензора модуляторов (разд. 4). Высокий ацетилхолин соответствует режиму бдительности: сенсорный вход усиливается, а обратные контекстные связи подавляются. Низкий ACh даёт приоритет апикальному NMDA-току — режим эндогенных ассоциаций, размышления и, в пределе, генеративного реплея во сне (разд. 7.3).

3.4 Сома с адаптивным порогом (ALIF)

Динамика субпорогового мембранного потенциала описывается классическим уравнением утечки:

$$\tau_m \frac{dV(t)}{dt} = -\big(V(t) - V_{\text{rest}}\big) + R_m\, I_{\text{soma}}(t), \tag{6}$$

а порог спайка — *адаптивным*, что принципиально отличает ALIF от наивного LIF:

$$\theta(t) = \theta_0 + \beta \cdot a(t), \qquad \tau_a \frac{da(t)}{dt} = -a(t) + s(t). \tag{7}$$

Каждый собственный спайк увеличивает переменную адаптации $a(t)$ и тем самым поднимает порог: часто стреляющий нейрон постепенно «устаёт» и требует всё более сильного входа. Адаптация — встроенный предохранитель от гипер-синхронизации: популяция не может раскачаться в эпилептиформный шторм, потому что каждый участник шторма сам себя гасит. Событие спайка определяется функцией Хевисайда

$$s(t) = \Theta\big(V(t) - \theta(t)\big) \tag{8}$$

с мгновенным сбросом потенциала; в реализации используется reset-by-subtraction, который, в отличие от жёсткого сброса, сохраняет градиентную информацию о величине превышения порога. Отдельный сервисный механизм — фазический сброс адаптации норадреналином: при всплеске surprise NE обнуляет накопленную усталость (уравнение (7)), возвращая «залипшей» популяции способность немедленно реагировать (см. разд. 4).

4 Нейрохимическая система: два контура регуляции

В системе строго разделены *быстрые ионотропные нейротрансмиттеры* — прямые токи миллисекундного масштаба — и *медленные метаботропные нейромодуляторы* — тензоры пластичности и энтропии, меняющие режим работы целых зон (рис. 3). Это разделение физиологично и одновременно инженерно: у двух контуров разные постоянные времени, разные аргументы и разные законы управления, поэтому смешивать их в одном механизме значило бы потерять управляемость обоих.

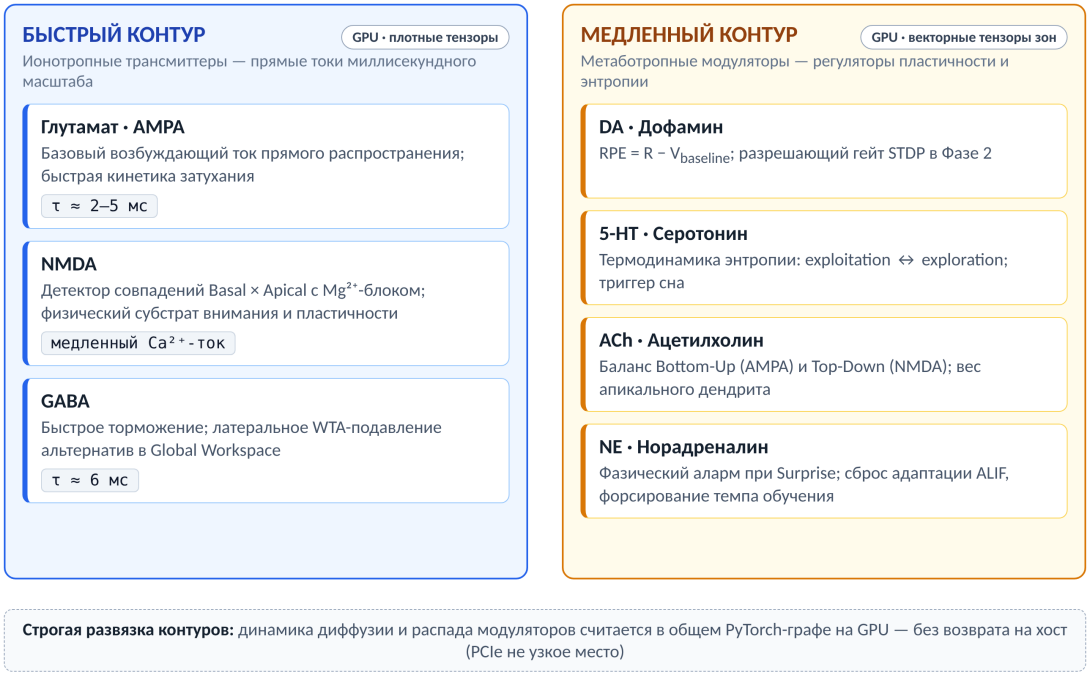


Рис. 3. Нейрохимический контур: быстрые ионотропные токи и медленные метаботропные модуляторы; оба считаются на GPU в общем вычислительном графе.

4.1 Быстрый контур: ионотропные трансмиттеры

Глутамат (AMPA) — базовый возбуждающий ток прямого распространения с быстрой кинетикой затухания ($\tau \approx 2$–5 мс), уравнение (3). **NMDA** — медленный кальциевый ток с потенциал-зависимым магниевым блоком, уравнение (4); он одновременно выступает физическим субстратом пластичности и направленного внимания, поскольку открывается только при совпадении входов. **GABA (ГАМК)** — базовое быстрое торможение. Оно реализовано через пул локальных тормозных интернейронов внутри зон и глобальный контур в Global Workspace. Механизм латерального торможения (WTA) подавляет альтернативы при активации доминантной популяции:

$$I_{\mathrm{GABA},\,i}(t) = \gamma_{\mathrm{lat}} \sum_{j \neq i} W_{ij}^{\mathrm{GABA}}\, s_j(t), \tag{9}$$

где γ_{lat} — общий коэффициент, а W^{GABA} — тормозная матрица латеральных связей. Именно этот механизм гарантирует отсутствие эпилептиформных спайковых штормов и формирует единственный когерентный фокус внимания в Global Workspace (разд. 2.4).

4.2 Медленный контур: метаботропные модуляторы

Ключевое архитектурное решение: модуляторы — это *не глобальные скаляры, а пространственные векторные тензоры на GPU*, привязанные к топологии зон. Иначе говоря, у каждой зоны — свой профиль нейрохимии, и «настроение» организма неоднородно по его объёму. Сводка ролей всех пяти агентов приведена в таблице 4.

Таблица 4. Нейрохимические агенты и их роли.

Агент	Контур	Роль в системе
Дофамин (DA)	медленный	ошибка предсказания награды RPE $= R(t) - V_{\text{baseline}}$; разрешающий ключ (гейт) закрепления синаптических следов в Фазе 2; закрепление полезных связей
Серотонин (5-HT)	медленный	регулятор термодинамической энтропии и стабильности: высокий уровень сжимает распределения (эксплуатация, шаблон), низкий — растит шум и исследовательское поведение; управляет давлением сна
Ацетилхолин (ACh)	медленный	балансировщик Bottom-Up и Top-Down: высокий уровень усиливает базальный AMPA-ток (бдительность к внешним данным), низкий даёт приоритет апикальному NMDA-току (эндогенные ассоциации, реплей)
Норадреналин (NE)	медленный	фазический аларм: выбрасывается при детекции всплеска Surprise в SON; локально сбрасывает адаптационные пороги ALIF, обнуляет залипание в локальных минимумах, кратковременно форсирует темп обучения
GABA-пул	быстрый	латеральное торможение: конкуренция зон в Global Workspace, подавление спайковых штормов, фильтрация шины Callosum

Пара DA/5-HT образует «качели» режимов обучения: дофамин разрешает закрепление только того, что уже произошло и было успешным, а серотонин определяет, насколько широко сеть ищет кандидатов на успех. ACh и NE замыкают на эти качели контекст: ацетилхолин выбирает источник информации (внешний или внутренний), норадреналин — момент, когда привычные правила перестали работать и надо переучиваться быстрее обычного. Все четыре модулятора — обучаемые тензоры со своей динамикой диффузии и распада, которая интегрирована в общий вычислительный граф на GPU (разд. 8); CPU не участвует в их пошаговом счёте.

5 Синапс с состоянием: две шкалы памяти и следы пригодности

5.1 Две шкалы синаптической памяти

Синапс NEUROWEAVE хранит больше, чем просто вес, — он несёт *состояние* с двумя временными шкалами. Долговременная память w_{slow} — медленно меняющийся базис, который обучается только в Фазе 1 локального обучения зон и хранит кристаллизованные навыки. Быстрая адаптация w_{fast} — оперативный буфер, который обучается через Reward-STDP в Фазе 2 и несёт контекстные, ситуативные коррекции. По окончании Фазы 1 с матрицы w_{slow} снимается контрольный снапшот W_{snapshot}: он служит якорем для регуляризации EWC и точкой возврата при консолидации (разд. 7.3). Эффективный вес синапса, участвующий в прямом распространении, — просто сумма $w_{\text{slow}} + w_{\text{fast}}$, при этом w_{fast} жёстко ограничен амплитудой и медленно затухает, что не даёт оперативной памяти «съесть» базис.

5.2 Следы пригодности и Tag-and-Capture: честная память $O(N)$

Трёхфакторная пластичность требует знать, *какие* синапсы недавно участвовали в активности, чтобы в момент прихода награды усилить именно их. Классическая запись — след пригодности на каждый синапс:

$$\Delta w_{\text{fast},\,ij}(t) = \eta \cdot \mathrm{DA}(t) \cdot \mathcal{E}_{ij}(t), \qquad \mathcal{E}_{ij}(t) = \bar{z}_{\text{pre},j}(t)\, s_{\text{post},i}(t) - s_{\text{pre},j}(t)\, \bar{z}_{\text{post},i}(t), \tag{10}$$

где $\bar{z}_{\text{pre}}$, $\bar{z}_{\text{post}}$ — экспоненциально затухающие следы пред- и постсинаптических спайков. Механизм Tag-and-Capture устроен так: если след пригодности живёт дольше задержки сигнала награды ($\tau_{\text{elig}} > \text{delay}$), то пришедший дофамин «захватывает» меченые синапсы и превращает их временное усиление в устойчивое изменение w_{fast}.

Здесь скрыта ловушка, устранённая в каноне v4.0. Матрица $\mathcal{E}_{ij}$ в каждый момент времени имеет ранг 1 (внешнее произведение двух векторов), но при *накоплении во времени* с затуханием τ_e она перестаёт иметь ранг 1 — и потребовала бы полной памяти $O(N^2)$ на каждый синаптический слой, что уничтожает заявленную экономию VRAM. Честное решение: хранить не матрицу, а два *вектора* следов на уровне самих нейронов — короткий спайковый след $\bar{s}$ и длинный след пригодности $\bar{e}$:

$$\frac{d\bar{e}}{dt} = -\frac{\bar{e}}{\tau_e} + \bar{s}, \tag{11}$$

а в момент прихода положительного дофамина обновлять w_{fast} *напрямую внешним произведением векторов* размерности $O(N)$, без хранения матрицы $\mathcal{E}_{ij}$ вообще:

$$\Delta w_{\text{fast}} = \eta\, \mathrm{DA}\big(A_+\, \bar{e}_{\text{pre}} \otimes s_{\text{post}} - A_-\, s_{\text{pre}} \otimes \bar{e}_{\text{post}}\big). \tag{12}$$

Слагаемые читаются как LTP (долговременное потенцирование: длинный предсинаптический след встречает свежий постсинаптический спайк) и LTD (депрессия: свежий пресинаптический спайк встречает затухающий постсинаптический след). Именно эта схема реализована методом `update_traces_and_plasticity()` класса `StatefulSynapse` через операцию тензорного умножения `einsum` (листинг 4).

5.3 Защита от катастрофического забывания

Сборка организма в Фазе 2 обязана не разрушить то, чему зоны научились в изоляции. Первая линия защиты — *жёсткая заморозка*: внутренние веса w_{slow} зон выключаются из градиентного счёта целиком (`requires_grad=False`); обучаются только w_{fast}, шины SON и Corpus Callosum. Вторая линия — *эластичная консолидация весов* (EWC): если в отдельных критических местах микро-дрейф w_{slow} всё же допускается, к лоссу добавляется квадратичный штраф Фишера, подтягивающий веса к снапшоту Фазы 1:

$$\mathcal{L}_{\text{EWC}} = \sum_k F_k \big(w_k - W_{\text{snapshot},k}\big)^2. \tag{13}$$

Третья линия — *запрет негативного разрушения*: сигнал подкрепления обрезается снизу нулём,

$$\mathrm{DA}(t) = \max\big(0,\ R(t) - V_{\text{baseline}}\big), \tag{14}$$

так что сеть учится *только на успехах*: неудачи не штрафуют и не размывают уже собранные связи, а лишь оставляют след непригодным для захвата. Практика STDP показывает, что без этой асимметрии отрицательное подкрепление быстро расшатывает и разрушает собранную топологию.

6 SON: нейроны само-наблюдения и межзональная шина

6.1 Активная обучаемая шина

SON (Self-Observation Neurons) — это не пассивный диагностический логгер, а *активная обучаемая межзональная шина связи*. В каждой зоне живёт ансамбль из 16 специализированных нейронов, которые выполняют четыре функции. *Компрессия*: SON-энкодер сжимает высокоразмерное пространственно-временное состояние зоны в компактный латентный спайковый вектор

$$z_{\text{SON}}(t) = f_{\text{enc}}\big(s_{\text{Zone}}(t-k\ldots t)\big). \tag{15}$$

Маршрутизация: вектор z_{SON} проходит через буферизованную шину Corpus Callosum с настраиваемой задержкой 5–10 шагов и разреженным GABA-фильтром. *Декодирование*: приёмный узел SON в зоне-адресате транслирует латентный вектор в пространственно распределённый ток на *апикальные* дендриты нейронов-получателей:

$$I_{\text{apical}}(t) = W_{\text{dec}} \cdot z_{\text{SON}}(t - \text{delay}). \tag{16}$$

Существенно, что SON-путь замыкается именно на апикальный компартмент: через NMDA-гейт уравнения (4) пересланная информация способна *модулировать* реакцию зоны-получателя, но не способна запустить её в отсутствие собственного входа — межзонное влияние принципиально контекстное.

6.2 Метакогниция и детекция Surprise

Второе назначение SON — самонаблюдение организма. Ансамбль локально рекуррентно предсказывает собственное следующее состояние $\hat{z}_{\text{SON}}(t+1)$, и вектор ошибки этого предсказания служит измерителем неожиданности:

$$\text{Surprise}(t) = \big\| z_{\text{SON}}(t) - \hat{z}_{\text{SON}}(t) \big\|^2. \tag{17}$$

Низкий Surprise означает, что зона находится в предсказуемом режиме и её динамика хорошо объяснена собственной моделью; высокий Surprise — сигнал, что зона столкнулась с тем, чего не может объяснить.

6.3 Гейтинг, экономия энергии и саморефлексия

Измеритель Surprise управляет двумя экономическими решениями. Если неожиданность низка, SON *молчит*: не расходует спайки, не занимает шину, не беспокоит соседей — режим Zero-Compute, за счёт которого событийная сеть действительно экономит энергию. Если Surprise высок, SON экспортирует латентное состояние, отправляет приоритетный пакет через мозолистое тело и триггерит фазический выброс норадреналина, который сбрасывает залипание адаптации в своей и соседних зонах. Наконец, именно SON решает, *какую* информацию передать в другие зоны, — и в этом смысле шина создаёт саморефлексию: сеть «чувствует» собственные внутренние процессы и может сообщать о них другим частям себя. Это самый близкий к сознанию примитив в архитектуре: компактное внутреннее описание состояния, доступное другим подсистемам по запросу и в момент неожиданности.

7 Трёхфазная методология обучения

Обучение организма разбито на три фазы с жёсткими контрактами на каждой границе (рис. 4). Фазы отличаются тем, *какие* веса открыты для изменения, *какой* сигнал обучения используется и *какие* связи вообще существуют. Общая логика: сначала каждый орган учится работать в одиночку, затем органы собираются в организм через дофаминовое подкрепление, и, наконец, организм периодически спит, переваривая накопленный опыт.

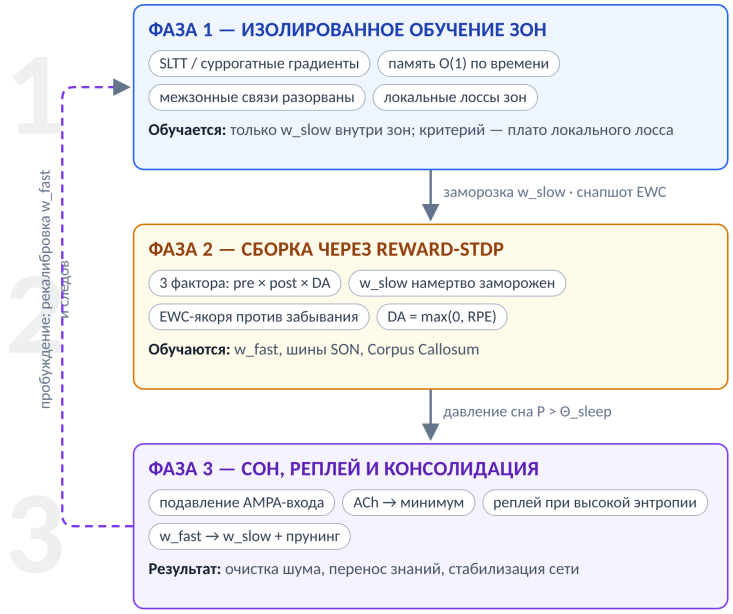


Рис. 4. Жизненный цикл обучения: изолированные зоны → сборка через Reward-STDP → сон и консолидация → пробуждение с рекалибровкой.

7.1 Фаза 1: изолированное обучение зон

Цель фазы — научить каждую зону выполнять свою задачу без всякой внешней помощи. *Строгая изоляция*: любые межзонные синапсы, проекции мозолистого тела и кросс-коммуникации физически разорваны — маскированы нулями; входные и выходные шины зон замкнуты на их локальные задачи (ур. (2)–(1)). *Обучаемый субстрат*: модифицируются исключительно внутренние матрицы долговременных синапсов w_{slow} конкретной зоны. *Метод* — пространственно-временное обучение на суррогатных градиентах (SLTT): прямое прохождение — ступенька Хевисайда (8), обратное — псевдопроизводная, например FastSigmoid

$$\frac{\partial s}{\partial V} \approx \frac{1}{\left(1 + k\,|V - \theta|\right)^2} \tag{18}$$

или Arctan с тяжёлыми хвостами, полезный для стохастичного правого полушария:

$$\frac{\partial s}{\partial V} \approx \frac{\alpha/2}{1 + \left(\frac{\pi}{2}\,\alpha\,(V - \theta)\right)^2}. \tag{19}$$

Канон v4.0 — истинная память $O(1)$ по времени. Чтобы PyTorch Autograd не строил граф вглубь временных шагов (BPTT) и не съедал всю VRAM, на каждом такте t выполняется `detach()` мембранного потенциала $V(t-1)$ и переменной адаптации $a(t-1)$; суррогатный градиент проходит строго через текущие синаптические токи. История состояний перестаёт быть частью графа, и потребление памяти перестаёт расти с длиной последовательности. Именно так работает метод `detach_state()` класса `MultiCompartmentALIF` (листинг 3).

Критерий завершения — конвергенция локального лосса до плато ($\Delta\mathcal{L} < \epsilon$). По завершении снимается неизменяемый контрольный снапшот $W_{\text{snapshot}} = w_{\text{slow}}^{\text{Phase1}}$, используемый далее EWC (ур. (13)) и консолидацией сна.

7.2 Фаза 2: межзонная сборка через Reward-STDP

Цель фазы — связать обученные зоны в единую систему, не повредив их навыкам. Правило заморозки: внутренние веса w_{slow} зон отключаются от градиентов, и разрешается адаптация только трёх подсистем: матриц быстрой пластичности w_{fast}, синапсов Corpus Callosum и проекций между шинами SON. Сигнал обучения — трёхфакторное правило «пре-спайк $\times$ пост-спайк $\times$ награда» (10) с дофаминовым гейтом (14): след пригодности (11) метит синапс-кандидата, а положительная ошибка предсказания награды захватывает меченые следы и превращает их в устойчивый прирост w_{fast} внешним произведением (12). Три линии защиты от забывания (заморозка, EWC, только положительная награда) разобраны в разд. 5. Верификация фазы — валидация устойчивости задач Фазы 1 при включённой адаптации Фазы 2: синтаксический навык Зоны A не должен деградировать при дообучении межзонных шин.

7.3 Фаза 3: сон, реплей и консолидация

Третья фаза — медленный контур гомеостаза. *Давление сна* накапливается не по часам, а как интеграл нагрузки организма — общей спайковой активности и истощения пула серотонина:

$$\frac{dP_{\text{sleep}}}{dt} = \kappa_1 \sum s(t) - \kappa_2 \,[\text{5-HT}](t). \tag{20}$$

При достижении порога $P_{\text{sleep}} > \Theta_{\text{sleep}}$ медленный контур CPU (разд. 8) переводит организм в режим сна, и протокол выполняет пять шагов:

1. отключаются внешние сенсорные базальные входы (AMPA-подавление);
2. ацетилхолин переводится в минимальный режим — доминируют апикальные эндогенные токи (низкий ACh, см. (5));
3. генеративный реплей: сеть спонтанно генерирует последовательности спайков из латентных состояний при повышенной энтропии (за счёт модуляции 5-HT) и дообучается на собственном материале;
4. медленная консолидация: устойчиво подтвердившиеся паттерны w_{fast} переносятся в кристаллическую матрицу w_{slow}

$$w_{\text{slow}} \leftarrow w_{\text{slow}} + \lambda_{\text{consol}} \cdot \text{Sign}\big(w_{\text{fast}}\big) \odot \big|w_{\text{fast}}\big|; \tag{21}$$

5. обнуляются и рекалибруются буферы w_{fast} и следы пригодности; слабоактивные синаптические пути структурно прореживаются (прунинг).

Итог цикла — очистка шума, перенос знаний из оперативной памяти в долговременную и стабилизация сети; после пробуждения организм продолжает работу с чистыми буферами, сохраняя консолидированный опыт.

8 Вычислительная архитектура: Dual-Channel Engine

8.1 Принцип: работающая система, а не академическая симуляция

Цель проекта — не демонстрационная симуляция для статьи, а работающая система на обычном оборудовании. Отсюда центральное правило проектирования: все горячие вычисления — на GPU в парадигме плотных тензоров, всё остальное — асинхронно и редко. Распределение ответственности между контурами зафиксировано в таблице 5.

8.2 Почему модуляторы не живут на CPU

Выносить вычисления нейромодуляторов или отдельных мембран на CPU недопустимо: межшинный обмен PCIe уничтожит пропускную способность — пересылка тензорных срезов на каждый такт обойдётся дороже самих вычислений. Поэтому динамика *всех* модуляторов считается на

Таблица 5. Разделение ответственности быстрого и медленного контуров.

Контур	Такт	Обязанности
Быстрый (GPU, PyTorch-граф)	каждый dt	мембранные потенциалы $V(t)$ и адаптивные пороги $\theta(t)$; спайковые маски $s(t)$; ионотропные токи AMPA / NMDA / GABA; *все* векторы нейромодуляторов DA, 5-HT, ACh, NE как GPU-тензоры с динамикой диффузии и распада в общем графе; кольцевые буферы задержек в VRAM
Медленный (CPU, асинхронно)	$K \approx 100$–500 шагов	агрегированные скалярные метрики (суммарная энтропия, общая частота спайков, статус сна); макро-решения: переключение фаз бодрствование $\leftrightarrow$ сон, корректировка глобального learning-rate scheduler, логирование телеметрии на диск и дашборд

GPU, внутри общего PyTorch-графа, без единого возврата на хост; CPU лишь изредка забирает уже агрегированные скаляры и возвращает редкие макро-команды (смена фазы, коэффициент LR). Такая асимметрия сохраняет всю нейрохимию организма «горячей», оставляя холодным только мета-контроль.

8.3 Инженерные оптимизации

Три оптимизации фиксируются спецификацией. Первая — *Dense Tensors вместо разреженных операций*: для сетей до ~ 5000 нейронов плотные матрицы на обычном GPU быстрее, чем разреженные (cuSPARSE) — разреженность топологии Callosum реализуется масками, а не разреженным хранением. Вторая — *CUDA Graphs*: захват цепочки ядер в граф устраняет оверхед запуска на каждом такте, что критично при шаге симуляции в один миллисекундный тик. Третья — *квантизация*: int8-активности и тернарные веса $(-1, 0, +1)$ в некритичных путях дают кратную экономию памяти и полосы пропускания. Все три оптимизации относятся к Этапу 5 дорожной карты (разд. 10) и не должны преждевременно усложнять ядро Этапов 0–2.

9 Реализация Этапа 0: фундаментальное ядро

Ниже приведён полный, выверенный код четырёх базовых модулей системы и верификационных тестов к ним — строго по канонической спецификации настоящего документа. Ядро размещается в `src/spiker/core/` и образует самостоятельный нижний слой проекта: ни один модуль не зависит от топологии, модуляторов или фаз обучения. Для каждого модуля указаны контрактные требования, которые проверяются юнит-тестами (разд. 9.5); код снабжён русскими комментариями и является эталонным стилем для всех последующих этапов.

9.1 `surrogate.py`: суррогатные градиенты

Модуль реализует две калиброванные псевдопроизводные спайка — FastSigmoid (18) для левого полушария и Arctan (19) с тяжёлыми хвостами для стохастичного правого. Параметр крутизны сознательно вынесен в аргумент: в дальнейшем он будет модулироваться серотонином 5-HT, повышающим или понижающим «резкость» обучения в зависимости от режима организма (разд. 4).

Листинг 1. Модуль `src/spiker/core/surrogate.py`: суррогатные градиенты спайка.

```python
# src/spiker/core/surrogate.py
import torch
import torch.nn as nn
```

```python
import math


class FastSigmoidSpike(torch.autograd.Function):
    """
    Суррогатный градиент на базе Fast Sigmoid:
    Forward: ступенчатая функция Хевисайда Theta(x)
    Backward: 1 / (1 + k * |x|)^2
    """
    @staticmethod
    def forward(ctx, x: torch.Tensor, slope: float = 10.0) -> torch.Tensor:
        ctx.save_for_backward(x)
        ctx.slope = slope
        return (x >= 0.0).to(x.dtype)

    @staticmethod
    def backward(ctx, grad_output: torch.Tensor):
        (x,) = ctx.saved_tensors
        slope = ctx.slope
        denom = 1.0 + slope * torch.abs(x)
        grad_x = grad_output / (denom * denom)
        return grad_x, None


class ArctanSpike(torch.autograd.Function):
    """
    Суррогатный градиент на базе Arctan:
    Backward: (alpha / 2) / (1 + (pi/2 * alpha * x)^2)
    Отличается более тяжелыми хвостами, что полезно для Правого полушария.
    """
    @staticmethod
    def forward(ctx, x: torch.Tensor, alpha: float = 2.0) -> torch.Tensor:
        ctx.save_for_backward(x)
        ctx.alpha = alpha
        return (x >= 0.0).to(x.dtype)

    @staticmethod
    def backward(ctx, grad_output: torch.Tensor):
        (x,) = ctx.saved_tensors
        alpha = ctx.alpha
        factor = (math.pi / 2.0) * alpha * x
        grad_x = grad_output * (alpha / 2.0) / (1.0 + factor * factor)
        return grad_x, None


def spike_fn(x: torch.Tensor, mode: str = "fast_sigmoid", slope: float = 10.0) ->
    torch.Tensor:
    if mode == "fast_sigmoid":
        return FastSigmoidSpike.apply(x, slope)
    elif mode == "arctan":
        return ArctanSpike.apply(x, slope)
    else:
        raise ValueError(f"Unknown surrogate mode: {mode}")
```

9.2 `transmitters.py`: ионотропные токи AMPA, NMDA, GABA

Модуль вычисляет и фильтрует во времени три ионотропных тока и реализует главное биофизическое требование спецификации — снятие магниевого блока только при базальной деполяризации, уравнение (4). Обратите внимание на калибровку сигмоиды: сдвиг `mg_zero_baseline` гарантирует, что при нулевом базальном токе гейт строго равен нулю, а не «маленькому остатку» —

иначе детектор совпадений протекал бы и тест апикальной тишины (разд. 9.5) не проходился бы. Ацетилхолиновая модуляция проводимостей соответствует канонической формуле сомы (5).

Листинг 2. Модуль `src/spiker/core/transmitters.py`: кинетика ионотропных токов и NMDA-гейт.

```python
# src/spiker/core/transmitters.py
import torch
import torch.nn as nn
from typing import Optional


class IonotropicTransmitters(nn.Module):
    """
    Вычисляет ионотропные токи (AMPA, NMDA, GABA) и их фильтрацию во времени.
    Реализует физиологический детектор совпадений через снятие Mg2+ блока.
    """
    def __init__(
        self,
        num_neurons: int,
        dt: float = 1.0,
        tau_ampa: float = 4.0,
        tau_nmda: float = 40.0,
        tau_gaba: float = 6.0,
        theta_mg: float = 0.25,
        k_mg: float = 12.0,
        alpha_nmda: float = 1.5,
        gamma_ach: float = 0.5,
    ):
        super().__init__()
        self.num_neurons = num_neurons
        self.dt = dt
        self.theta_mg = theta_mg
        self.k_mg = k_mg
        self.alpha_nmda = alpha_nmda
        self.gamma_ach = gamma_ach

        # Коэффициенты экспоненциального затухания проводимостей (Exact integration)
        self.register_buffer("decay_ampa", torch.exp(torch.tensor(-dt / tau_ampa)))
        self.register_buffer("decay_nmda", torch.exp(torch.tensor(-dt / tau_nmda)))
        self.register_buffer("decay_gaba", torch.exp(torch.tensor(-dt / tau_gaba)))

        # Состояния синаптических токов
        self.i_ampa: Optional[torch.Tensor] = None
        self.i_nmda_raw: Optional[torch.Tensor] = None
        self.i_gaba: Optional[torch.Tensor] = None

    def reset_state(self, batch_size: int, device: torch.device, dtype: torch.dtype
    = torch.float32):
        shape = (batch_size, self.num_neurons)
        self.i_ampa = torch.zeros(shape, device=device, dtype=dtype)
        self.i_nmda_raw = torch.zeros(shape, device=device, dtype=dtype)
        self.i_gaba = torch.zeros(shape, device=device, dtype=dtype)

    def detach_state(self):
        """Для O(1) памяти SLTT в Фазе 1: отсекаем историю градиентов."""
        if self.i_ampa is not None:
            self.i_ampa = self.i_ampa.detach()
            self.i_nmda_raw = self.i_nmda_raw.detach()
            self.i_gaba = self.i_gaba.detach()

    def forward(
        self,
        raw_basal: torch.Tensor,
```

```
        raw_apical: torch.Tensor,
        raw_gaba: torch.Tensor,
        ach_level: Optional[torch.Tensor] = None,
    ) -> torch.Tensor:
        """
        Параметры:
            raw_basal: [Batch, N] — мгновенный входной ток W_AMPA * s_pre(t)
            raw_apical: [Batch, N] — мгновенный контекстный ток W_apical * s_context(t)
            raw_gaba: [Batch, N] — мгновенный тормозной ток W_GABA * s_inh(t)
            ach_level: [Batch, N] или [Batch, 1] в диапазоне [-1, 1] уровень(Ацетилхолина)
        Возвращает:
            i_soma: [Batch, N] — итоговый ток, поступающий в сому нейрона
        """
        if self.i_ampa is None or self.i_ampa.shape != raw_basal.shape:
            self.reset_state(raw_basal.shape[0], raw_basal.device, raw_basal.dtype)

        # 1. Кинетика синаптических фильтров
        self.i_ampa = self.i_ampa * self.decay_ampa + raw_basal
        self.i_nmda_raw = self.i_nmda_raw * self.decay_nmda + raw_apical
        self.i_gaba = self.i_gaba * self.decay_gaba + raw_gaba

        # 2. Математика NMDAгейтинга- снятие( Mg2+ блока только при базальной деполяризации)
        # Используем калиброванную сигмоиду со сдвигом так, чтобы при i_ampa == 0 гейт был строго ~0
        mg_unblock = torch.sigmoid(self.k_mg * (self.i_ampa - self.theta_mg))
        mg_zero_baseline = torch.sigmoid(torch.tensor(-self.k_mg * self.theta_mg, device=raw_basal.device))
        mg_gate = torch.clamp((mg_unblock - mg_zero_baseline) / (1.0 - mg_zero_baseline), min=0.0, max=1.0)

        i_nmda_effective = mg_gate * torch.relu(self.i_nmda_raw)

        # 3. Модуляция Ацетилхолином (ACh: баланс Bottom-Up vs Top-Down)
        if ach_level is not None:
            basal_gain = 1.0 + self.gamma_ach * ach_level
            apical_gain = 1.0 - self.gamma_ach * ach_level
        else:
            basal_gain = 1.0
            apical_gain = 1.0

        # 4. Итоговый ток в сому
        i_soma = (basal_gain * self.i_ampa) + (self.alpha_nmda * apical_gain * i_nmda_effective) - self.i_gaba
        return i_soma
```

9.3 `neuron.py`: мульти-компартментный ALIF-нейрон

Класс `MultiCompartmentALIF` собирает компартментную схему рис. 2: дендритные токи считает встроенный `IonotropicTransmitters`, сома интегрирует мембранный потенциал (6) с адаптивным порогом (7) и порождает спайки (8) через `spike_fn`. Модуляторные входы реализуют канонические правила: NE подавляет накопленную адаптацию (сброс «усталости» при surprise), 5-HT управляет амплитудой стохастического шума мембраны (низкий серотонин — рост энтропии до трёхкратного), флаг `slt_mode` включает пошаговый `detach` для $O(1)$-памяти Фазы 1. Сброс потенциала — вычитанием: он сохраняет градиентную информацию о превышении порога.

Листинг 3. Модуль `src/spiker/core/neuron.py`: мульти-компартментный ALIF-нейрон.

```python
# src/spiker/core/neuron.py
import torch
import torch.nn as nn
from typing import Optional, Tuple
from .surrogate import spike_fn
from .transmitters import IonotropicTransmitters


class MultiCompartmentALIF(nn.Module):
    """
    Трехсегментный нейрон Базальный( дендрит, Апикальный дендрит, Сома ALIF).
    Интегрирует AMPA/NMDA/GABA токи и управляется метаботропными модуляторами (ACh,
    NE, 5-HT).
    """
    def __init__(
        self,
        num_neurons: int,
        dt: float = 1.0,
        tau_m: float = 10.0,       # 5-10 мс для Левого, 25-50 мс для Правого
    полушария
        tau_a: float = 100.0,      # Константа времени адаптации порога
        v_rest: float = 0.0,
        theta_0: float = 1.0,      # Базовый порог спайка
        beta_adapt: float = 0.15,  # Сила шага адаптации порога
        r_m: float = 1.0,
        sigma_noise: float = 0.001,# Базовый шум мембраны
        surrogate_mode: str = "fast_sigmoid",
        surrogate_slope: float = 10.0,
        **transmitter_kwargs,
    ):
        super().__init__()
        self.num_neurons = num_neurons
        self.dt = dt
        self.v_rest = v_rest
        self.theta_0 = theta_0
        self.beta_adapt = beta_adapt
        self.r_m = r_m
        self.sigma_noise = sigma_noise
        self.surrogate_mode = surrogate_mode
        self.surrogate_slope = surrogate_slope

        # Экспоненциальные константы мембраны и адаптации
        self.register_buffer("alpha_m", torch.exp(torch.tensor(-dt / tau_m)))
        self.register_buffer("alpha_a", torch.exp(torch.tensor(-dt / tau_a)))

        # Ионотропный аппарат дендритов
        self.transmitters = IonotropicTransmitters(num_neurons, dt=dt,
    **transmitter_kwargs)

        # Динамические состояния сомы
        self.v_mem: Optional[torch.Tensor] = None
        self.adapt: Optional[torch.Tensor] = None

    def reset_state(self, batch_size: int, device: torch.device, dtype: torch.dtype
    = torch.float32):
        shape = (batch_size, self.num_neurons)
        self.v_mem = torch.full(shape, self.v_rest, device=device, dtype=dtype)
        self.adapt = torch.zeros(shape, device=device, dtype=dtype)
        self.transmitters.reset_state(batch_size, device, dtype)

    def detach_state(self):
        """Обеспечивает O(1) потребление памяти по времени для SLTT Фаза( 1)."""
        if self.v_mem is not None:
```

```
            self.v_mem = self.v_mem.detach()
            self.adapt = self.adapt.detach()
            self.transmitters.detach_state()

    def forward(
        self,
        raw_basal: torch.Tensor,
        raw_apical: torch.Tensor,
        raw_gaba: torch.Tensor,
        ach: Optional[torch.Tensor] = None,
        ne: Optional[torch.Tensor] = None,
        serotonin_5ht: Optional[torch.Tensor] = None,
        sltt_mode: bool = True,
    ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """
        Один шаг симуляции dt.
        Возвращает: (spikes, v_mem, theta_current)
        """
        if self.v_mem is None or self.v_mem.shape != raw_basal.shape:
            self.reset_state(raw_basal.shape[0], raw_basal.device, raw_basal.dtype)

        if sltt_mode:
            # Отсекаем граф предыдущего шага для O(1) памяти
            self.detach_state()

        # 1. Вычисляем дендритные токи (AMPA + NMDA коинциденция - GABA)
        i_soma = self.transmitters(raw_basal, raw_apical, raw_gaba, ach_level=ach)

        # 2. Норадреналин (NE): фазический сброс залипания гасит( накопленную
усталость adapt)
        if ne is not None:
            ne_Suppression = torch.clamp(1.0 - ne, min=0.0, max=1.0)
            self.adapt = self.adapt * ne_Suppression

        # 3. Стохастический шум, модулируемый Серотонином (5-HT)
        # Высокий 5-HT ближе( к 1) -> подавление шума; Низкий 5-HT -> рост энтропии
        noise_scale = self.sigma_noise
        if serotonin_5ht is not None:
            noise_scale = self.sigma_noise * torch.clamp(1.0 - serotonin_5ht,
min=0.0, max=3.0)

        if self.training or noise_scale > 0.0:
            noise = torch.randn_like(self.v_mem) * noise_scale
        else:
            noise = 0.0

        # 4. Интегрирование мембранного потенциала сомы
        v_Candidate = (
            self.alpha_m * (self.v_mem - self.v_rest)
            + self.v_rest
            + (1.0 - self.alpha_m) * self.r_m * i_soma
            + noise
        )

        # 5. Вычисление адаптивного порога и генерация спайка
        theta_t = self.theta_0 + self.beta_adapt * self.adapt
        v_shifted = v_Candidate - theta_t
        spikes = spike_fn(v_shifted, mode=self.surrogate_mode,
slope=self.surrogate_slope)

        # 6. Сброс потенциала (Reset-by-subtraction сохраняет градиентную информацию
о превышении)
        self.v_mem = v_Candidate - spikes * (theta_t - self.v_rest)
```

```
        # 7. Обновление переменной адаптации порога
        self.adapt = self.alpha_a * self.adapt + spikes.detach()

        return spikes, self.v_mem, theta_t
```

9.4 `synapse.py`: синапс с состоянием и памятью $O(N)$

Класс `StatefulSynapse` несёт две шкалы памяти (w_{slow} как обучаемый параметр Фазы 1 и w_{fast} как буфер Фазы 2 с снапшотом для EWC) и реализует честную факторизацию следов пригодности разд. 5.2: хранятся четыре *вектора* ($\bar{z}_{\text{pre}}, \bar{z}_{\text{post}}$ и $\bar{e}_{\text{pre}}, \bar{e}_{\text{post}}$) размерности $O(N)$, а обновление весов при приходе положительного дофамина выполняется парой свёрток `einsum` — внешними произведениями ранга 1, без какой-либо матрицы $\mathcal{E}_{ij}$. Метод `freeze_for_phase2` фиксирует контракт Фазы 2: w_{slow} выключается из Autograd и снимается снапшот.

Листинг 4. Модуль `src/spiker/core/synapse.py`: синапс с состоянием и $O(N)$-следами.

```
# src/spiker/core/synapse.py
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Optional
import math


class StatefulSynapse(nn.Module):
    """
    Синаптическая матрица с состоянием:
    - w_slow: долговременная память обучается( суррогатным градиентом в Phase 1)
    - w_fast: быстрая пластичность обучается( через 3-factor Reward-STDP в Phase 2)
    - Честная O(N) память следов: хранит векторы следов размера [Batch, N_pre] и
    [Batch, N_post]
      вместо матрицы [Batch, N_post, N_pre].
    """
    def __init__(
        self,
        in_features: int,
        out_features: int,
        dt: float = 1.0,
        tau_pre: float = 20.0,
        tau_post: float = 20.0,
        tau_elig: float = 100.0,   # Окно памяти следа для Tag-and-Capture
        w_fast_max: float = 0.5,
        w_fast_decay: float = 0.9995,
    ):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.dt = dt
        self.w_fast_max = w_fast_max
        self.w_fast_decay = w_fast_decay

        # Долговременные веса (Phase 1)
        self.w_slow = nn.Parameter(torch.empty(out_features, in_features))
        nn.init.kaiming_uniform_(self.w_slow, a=math.sqrt(5))

        # Быстрые веса (Phase 2) - не участвуют в стандартном autograd оптимизаторе
        self.register_buffer("w_fast", torch.zeros(out_features, in_features))
        # Снапшот для EWC
        self.register_buffer("w_snapshot", torch.zeros(out_features, in_features))
```

```
43
44          # Константы затухания следов O(N)
45          self.register_buffer("decay_pre", torch.exp(torch.tensor(-dt / tau_pre)))
46          self.register_buffer("decay_post", torch.exp(torch.tensor(-dt / tau_post)))
47          self.register_buffer("decay_elig", torch.exp(torch.tensor(-dt / tau_elig)))
48
49          # Векторы следов размера O(N_pre) и O(N_post)
50          self.z_pre: Optional[torch.Tensor] = None
51          self.z_post: Optional[torch.Tensor] = None
52          self.e_pre: Optional[torch.Tensor] = None
53          self.e_post: Optional[torch.Tensor] = None
54
55      def reset_state(self, batch_size: int, device: torch.device, dtype: torch.dtype
    = torch.float32):
56          self.z_pre = torch.zeros((batch_size, self.in_features), device=device,
    dtype=dtype)
57          self.z_post = torch.zeros((batch_size, self.out_features), device=device,
    dtype=dtype)
58          self.e_pre = torch.zeros((batch_size, self.in_features), device=device,
    dtype=dtype)
59          self.e_post = torch.zeros((batch_size, self.out_features), device=device,
    dtype=dtype)
60
61      def freeze_for_phase2(self):
62          """Замораживает w_slow и сохраняет снапшот по окончании Phase 1."""
63          self.w_slow.requires_grad_(False)
64          self.w_snapshot.copy_(self.w_slow.data)
65
66      def forward(self, s_pre: torch.Tensor) -> torch.Tensor:
67          """Вычисляет постсинаптический ток: (w_slow + w_fast) @ s_pre."""
68          w_total = self.w_slow + self.w_fast
69          return F.linear(s_pre, w_total)
70
71      @torch.no_grad()
72      def update_traces_and_plasticity(
73          self,
74          s_pre: torch.Tensor,
75          s_post: torch.Tensor,
76          da_reward: Optional[torch.Tensor] = None,
77          lr_fast: float = 0.01,
78          a_plus: float = 1.0,
79          a_minus: float = 0.8,
80      ):
81          """
82          Обновляет O(N) следы и применяет факторное3- правило Reward-STDP,
83          если пришел положительный дофаминовый сигнал (DA > 0).
84          """
85          if self.z_pre is None or self.z_pre.shape[0] != s_pre.shape[0]:
86              self.reset_state(s_pre.shape[0], s_pre.device, s_pre.dtype)
87
88          # 1. Быстрые синаптические следы спайков
89          self.z_pre = self.z_pre * self.decay_pre + s_pre
90          self.z_post = self.z_post * self.decay_post + s_post
91
92          # 2. Медленные следы пригодности (Eligibility Traces) ранга 1 — память
    строго O(N)
93          self.e_pre = self.e_pre * self.decay_elig + self.z_pre
94          self.e_post = self.e_post * self.decay_elig + self.z_post
95
96          # 3. Tag-and-Capture при наличии положительного Дофамина: DA(t) = max(0, RPE)
97          if da_reward is not None:
98              # Убеждаемся, что награда неотрицательна Запрет( негативного разрушения,
    п. 4.2)
```

```
            da_pos = torch.clamp(da_reward, min=0.0)
            if torch.any(da_pos > 0):
                # Внешнее произведение усредненное по батчу: [Out, In]
                # LTP: преслед- * постспайк- | LTD: постслед- * преспайк-
                mod_post = s_post * da_pos  # [Batch, Out]
                mod_e_post = self.e_post * da_pos  # [Batch, Out]

                ltp = torch.einsum("bo,bi->oi", mod_post, self.e_pre)
                ltd = torch.einsum("bo,bi->oi", mod_e_post, s_pre)

                batch_size = float(s_pre.shape[0])
                delta_w = lr_fast * (a_plus * ltp - a_minus * ltd) / batch_size

                self.w_fast.mul_(self.w_fast_decay).add_(delta_w)
                self.w_fast.clamp_(-self.w_fast_max, self.w_fast_max)
```

9.5 Верификационные тесты ядра

Тесты фиксируют два ключевых контракта спецификации. Первый — биофизический: *апикальный контекст сам по себе не может вызвать спайк*, слабый подпороговый базальный вход не пробивает порог в одиночку, но совпадение слабого базального с сильным апикальным даёт взрывную активацию (три сценария `test_nmda_coincidence_detection`). Второй — ресурсный: следы пригодности имеют строго векторную форму $O(N)$ (никаких матриц $[B, N_{\text{post}}, N_{\text{pre}}]$), а контракт Фазы 2 соблюдён буквально — w_{slow} не изменяется ни на один элемент, w_{fast} обновляется только при положительном DA.

Листинг 5. Тесты `tests/test_core_foundation.py`: NMDA-коинциденция и $O(N)$-следы.

```
# tests/test_core_foundation.py
import torch
from src.spiker.core.neuron import MultiCompartmentALIF
from src.spiker.core.synapse import StatefulSynapse


def test_nmda_coincidence_detection():
    """
    Проверка п. 1.1: Нейрон НЕ должен активироваться исключительно апикальным
    контекстом сверху, но контекст мультипликативно усиливает реакцию на базальный
    вход.
    """
    torch.manual_seed(42)
    neuron = MultiCompartmentALIF(
        num_neurons=4,
        tau_m=10.0,
        theta_0=1.0,
        sigma_noise=0.0,  # Отключаем шум для чистоты эксперимента
        theta_mg=0.25,
        alpha_nmda=2.0,
    )
    neuron.eval()

    batch_size = 1
    zero = torch.zeros(batch_size, 4)

    # Сценарий 1: Сильный апикальный ток (Top-Down), НО нулевой базальный вход
    (Bottom-Up)
    neuron.reset_state(batch_size, torch.device("cpu"))
    strong_apical = torch.full((batch_size, 4), 5.0)
    spikes_apical_only = []
    for _ in range(20):
```

```
        s, v, _ = neuron(raw_basal=zero, raw_apical=strong_apical, raw_gaba=zero)
        spikes_apical_only.append(s.sum().item())

    assert sum(spikes_apical_only) == 0.0, "Ошибка! Апикальный вход вызвал спайк без базального!"

    # Сценарий 2: Слабый подпороговый базальный вход БЕЗ апикальной поддержки
    neuron.reset_state(batch_size, torch.device("cpu"))
    weak_basal = torch.full((batch_size, 4), 0.22)
    spikes_basal_only = []
    for _ in range(20):
        s, v, _ = neuron(raw_basal=weak_basal, raw_apical=zero, raw_gaba=zero)
        spikes_basal_only.append(s.sum().item())

    assert sum(spikes_basal_only) == 0.0, "Подпороговый базальный вход не должен пробивать порог в одиночку."

    # Сценарий 3: Коинциденция Слабый( базальный + Апикальный контекст одновременно)
    neuron.reset_state(batch_size, torch.device("cpu"))
    spikes_coincidence = []
    for _ in range(20):
        s, v, _ = neuron(raw_basal=weak_basal, raw_apical=strong_apical, raw_gaba=zero)
        spikes_coincidence.append(s.sum().item())

    assert sum(spikes_coincidence) > 0.0, "Ошибка! NMDAкоинциденция- не сработала при совпадении входов!"


def test_synapse_o_n_traces_and_phase2_freeze():
    """Проверка O(N) следов и защиты w_slow в Фазе 2."""
    syn = StatefulSynapse(in_features=16, out_features=8)
    syn.freeze_for_phase2()

    assert not syn.w_slow.requires_grad, "w_slow должен быть заморожен в Phase 2!"

    s_pre = torch.ones(2, 16)
    s_post = torch.ones(2, 8)
    da = torch.tensor([[1.0], [1.0]])

    w_slow_before = syn.w_slow.clone()
    syn.update_traces_and_plasticity(s_pre, s_post, da_reward=da, lr_fast=0.05)

    # Проверяем размерность следов строго( O(N), никаких матриц [Batch, 8, 16])
    assert syn.e_pre.shape == (2, 16)
    assert syn.e_post.shape == (2, 8)
    # w_slow не изменился, а w_fast обновился
    assert torch.allclose(syn.w_slow, w_slow_before)
    assert torch.abs(syn.w_fast).sum() > 0.0
```

10 Дорожная карта и файловая структура проекта

10.1 Этапы 0–5

Дорожная карта проекта зафиксирована в таблице 6. Каждый этап заканчивается верифицируемым критерием готовности — юнит-тестом, экспериментом или измерением, — поэтому прогресс невозможно имитировать: либо тест изоляции зон проходит, либо Этап не закрыт. Этап 0 (ядро `core/`) уже реализован — его код приведён в разд. 9; Этапы 1–5 описаны с точностью до файлов и контрактов.

Таблица 6. Детализированная дорожная карта (Этапы 0–5).

Этап	Модули	Ключевые задачи и критерий готовности
Этап 0 — математическое ядро	`core/`: `surrogate.py`, `transmitters.py`, `neuron.py`, `synapse.py`	тензорные структуры мембран и синапсов; суррогатные градиенты FastSigmoid/Arctan с калиброванными крутизнами; мульти-компартментный ALIF с раздельной адресацией AMPA/NMDA/GABA; потенциал-зависимый Mg^{2+}-блок; синапс с w_{slow}/w_{fast} и факторизованным следом $O(N)$. *Критерий:* тесты NMDA-коинциденции и $O(N)$-следов зелёные (разд. 9.5).
Этап 1 — зоны и модуляторный контур	`neurochemistry.py`, `son.py`, `zone.py`	GPU-тензорная динамика DA, 5-HT, ACh, NE; обучаемый блок SON (шина сжатия, метакогнитивный предиктор, генератор Surprise); класс `Zone` — замкнутый рекуррентный анклав с пулами нейронов и интернейронов. *Критерий (`test_zone_isolated.py`):* Зона A обучается предсказанию токенов при физическом отсутствии связей с Зоной C; утечка градиентов и информации между зонами отсутствует; апикальный ток без базального не вызывает спайка.
Этап 2 — слом симметрии и макротопология	`callosum.py`, `hemisphere.py`, `workspace.py`	`CorpusCallosum`: кольцевой GPU-буфер задержек, разреженная топология, асимметричные веса; `LeftHemisphere`/`RightHemisphere` с фиксацией дивергенции (τ_m, lr, шум, серотониновый базис, табл. 2); `GlobalWorkspace` — латеральное GABA-торможение, Soft-WTA; интеграция шины SON-to-SON на апикальные дендриты. *Критерий:* тесты `test_symmetry_breaking.py` и `test_workspace_wta.py`.
Этап 3 — двухфазный контур обучения и EWC	`train_phase1.py`, `stdp.py`, `ewc.py`	пайплайн пакетного обучения изолированных зон на суррогатных градиентах с независимыми лоссами; модуль жёсткой фиксации весов и снятия матрицы Фишера; пайплайн Фазы 2 — активация межзонных проекций и обучение w_{fast} и шин SON через 3-факторный Reward-STDP с дофаминовым гейтом. *Критерий:* валидация устойчивости задач Фазы 1 при адаптации Фазы 2 (защита от забывания).
Этап 4 — медленный контур, гомеостаз, сон	`slow_loop.py`, `organism.py`, `sleep.py`	асинхронный CPU-контроллер: периодический опрос энтропии, триггер фазы сна, $K \approx 100$–500 шагов; протокол сна: подавление AMPA-входа, снижение ACh, модуляция 5-HT, автономный стохастический реплей, консолидация $w_{fast} \to w_{slow}$, структурный прунинг слабоактивных связей. *Критерий:* воспроизводимый цикл бодрствование–сон–пробуждение без деградации навыков.

Этап	Модули	Ключевые задачи и критерий готовности
Этап 5 — сенсорные интерфейсы, валидация, оптимизация	`temporal_encoder.py`, профилирование, CUDA Graphs	временной фазовый кодировщик токенов (burst-coding и latency-coding); комплексный эксперимент — адаптивное переключение поведения при инверсии правил без переобучения базового синтаксиса; захват быстрого контура в CUDA Graphs; квантизация int8 / тернарные веса. *Критерий:* эксперимент инверсии правил пройден; профиль производительности быстрого контура снят.

10.2 Файловая структура проекта

Полная структура пакета `Spiker/` приведена в листинге 6. Дерево отражает послойную организацию системы: ядро биофизики (`core/`), нейрохимия и метакогниция (`modulation/`), макротопология (`topology/`), сборка организма (`organism/`), контуры обучения (`learning/`) и сенсорные интерфейсы (`encoding/`); конфигурации полушарий вынесены в YAML, тесты покрывают каждый контракт спецификации отдельным файлом.

Листинг 6. Файловая структура проекта `Spiker/`.

```
Spiker/├──
 idea.md                        # Исходный философскоархитектурный- манифест v3.0├──
 start.md                       # Техническая спецификация и дорожная карта (v3.1)├──
 configs/│
   ├── organism_default.yaml      # Базовые параметры топологии и констант│
   ├── hemisphere_left.yaml       # Быстрые константы, малый шум, высокий 5-HT│
   └── hemisphere_right.yaml      # Медленные константы, стохастичность, низкий 5-HT├──
 src/│
   └── spiker/│
       ├── __init__.py│
       ├── core/                  # Микробиофизика- нейронов и синапсов│
       │   ├── __init__.py│
       │   ├── neuron.py          # Multi-Compartment ALIF (Soma, Basal, Apical)│
       │   ├── transmitters.py    # Быстрые токи: AMPA, NMDA (Mg2+ block), GABA│
       │   ├── synapse.py         # Stateful Synapse (w_slow, w_fast, O(N) traces)│
       │   └── surrogate.py       # Суррогатные градиенты (FastSigmoid, Arctan)│
       ├── modulation/            # Нейрохимический контур и метакогниция│
       │   ├── __init__.py│
       │   ├── neurochemistry.py  # GPUтензоры- модуляторов: DA, 5-HT, ACh, NE│
       │   └── son.py             # Self-Observation Neurons Шина(, Surprise, Top-Down)│
       ├── topology/              # Макротопология- и латеральная селекция│
       │   ├── __init__.py│
       │   ├── zone.py            # Функциональная изолированная зона│
       │   ├── hemisphere.py      # Полушария асимметричные( параметры)│
       │   ├── callosum.py        # Буферизованное асимметричное мозолистое тело│
       │   └── workspace.py       # Global Workspace (GABAлатеральное- торможение)│
       ├── organism/              # Полная сборка организма и контуры управления│
       │   ├── __init__.py│
       │   ├── organism.py        # SpikerOrganism (GPU Fast Loop)│
       │   └── slow_loop.py       # Асинхронный CPU Meta-Controller Гомеостаз(, Сон)│
       ├── learning/              # Двухфазное обучение и сон│
       │   ├── __init__.py│
       │   ├── stdp.py            # 3-Factor Reward-STDP + Tag-and-Capture│
       │   ├── ewc.py             # Матрица Фишера и Elastic Weight Consolidation│
       │   └── sleep.py           # Оффлайнконсолидация- w_fast -> w_slow и реплей│
       └── encoding/              # Сенсорные интерфейсы│
           ├── __init__.py│
           └── temporal_encoder.py # Преобразование токенов в спайковые поезда├──
 tests/                         # Строгая верификация и регрессионные тесты│
   ├── test_transmitters.py       # Тест NMDAкоинциденции- и GABAторможения-│
```

```
   ├── test_neuron.py              # Тест мультикомпартментной- динамики ALIF│
   ├── test_synapse.py             # Тест факторизации следов пригодности O(N)│
   ├── test_zone_isolated.py       # Изолированный тест Phase 1: Zone A не знает о Zone C│
   ├── test_son_bus.py             # Тест кодированиядекодирования/ шины SON│
   ├── test_symmetry_breaking.py   # Проверка дивергенции левого и правого полушарий│
   └── test_workspace_wta.py       # Тест латерального подавления спайковых штормов└──
 main.py                         # Демонстрационная точка входа симуляции
```

10.3 Непосредственный план перехода к реализации

Для команды, продолжающей работу с этим документом, последовательность действий следующая. Сначала — создание скелета директорий и модулей по структуре листинга 6; фундаментальное ядро `core/` при этом уже готово (разд. 9). Затем — верификация ядра юнит-тестами `test_transmitters.py` и `test_neuron.py` с фиксацией двух инвариантов: отсутствия ложных спайков при наличии только апикального входа и экспоненциального затухания/накопления следов пластичности. Далее работа продолжается по Этапу 1 (разд. 14).

11 Ожидаемые свойства системы

Нешаблонный выход. Благодаря асинхронности задержек, усталости нейронов (адаптивные пороги) и внутренней «химии» модуляторов, сеть выдаёт разные ответы в зависимости от своего «настроения» и контекста. Это не баг, а проектная характеристика: организм с разным уровнем серотонина, разной историей недавней активности и разным фокусом Global Workspace физически не может выдать две идентичные генерации. Шаблонность ответа становится управляемой осью, а не константой архитектуры.

Непрерывное обучение без деградации. Система не деградирует со временем: трёхслойная защита от забывания (заморозка базиса, EWC, только положительная награда) плюс периодическая консолидация сна позволяют адаптироваться к новым данным «на лету», не теряя старых навыков. Оперативная память w_{fast} принимает ситуативные коррекции, а перенос подтвердившегося опыта в w_{slow} происходит только в безопасном офлайн-режиме сна.

Энергоэффективность. Благодаря гейтингу SON и событийности, сеть тратит вычисления только тогда, когда происходит что-то новое или важное. Предсказуемые режимы не генерируют ни спайков шины, ни тиков обучения; латеральное торможение подавляет бесполезную параллельную активность; квантизация и тернарные веса на верхних этапах сокращают стоимость каждого оставшегося тика.

> **Итоговая формула проекта.** Это не фреймворк. Это цифровой организм, который учится, спит, конкурирует внутри себя и адаптируется. Мы строим фундамент для ИИ нового поколения — и фундамент этот начинается с честной биофизики одного нейрона.

12 Аналитические приложения

12.1 Позиционирование: NEUROWEAVE против LLM и классических SNN

Сводное сравнение парадигм по десяти осям приведено в таблице 7. Её назначение — не «победить» трансформеры (это разные ниши), а точно зафиксировать, за счёт каких проектных решений NEUROWEAVE претендует на класс задач, недоступный статическим аппроксиматорам: непрерывная адаптация, устойчивость к забыванию, внутренние состояния и энергоэффективная событийность.

Таблица 7. Сравнение парадигм: трансформеры, классические SNN-фреймворки и NEUROWEAVE.

Ось	Трансформер (LLM)	Классический SNN-фреймворк	NEUROWEAVE
Роль спайков	отсутствуют	декорация над плотными матрицами	фундамент вычислительного графа
Обучение и инференс	разные фазы, веса заморожены	совмещены частично, чаще BPTT	единый процесс, обучение в момент генерации
Память	только в весах, между сессиями пусто	активации + веса	многошкальная: следы $\to w_{\text{fast}} \to w_{\text{slow}}$ + сон-консолидация
Пластичность	глобальный backprop	локальный STDP (зачастую без гарантий стабильности)	локальная: SLTT в зонах + 3-факторный Reward-STDP со сборкой
Защита от забывания	внешние средства (реплейер, дистилляция)	как правило, отсутствует	встроена: заморозка + EWC + только положительная награда
Нейромодуляция	нет (либо глобальный скаляр LR)	редко, глобальные скаляры	векторные тензоры DA/5-HT/ACh/NE на GPU, привязаны к зонам
Внимание	Soft-Attention над токенами	обычно отсутствует	физическое: NMDA-коинциденция + GABA-конкуренция в Workspace
Асимметрия вычислителей	однородные слои	однородные слои	полушария со сломом симметрии (τ_m, шум, 5-HT, LR)
Гомеостаз	нет	редко (homeostasis порогов)	встроен: ALIF-адаптация, давление сна, реплей, прунинг
Целевой режим работы	офлайн-обучение, онлайн-инференс	симуляция	непрерывная жизнь: бодрствование $\to$ сборка $\to$ сон

12.2 Матрица рисков и механизмы митигации

Каждый риск проекта закрыт механизмом, уже описанным в архитектуре; сводка приведена в таблице 8. Она же служит чек-листом при проектировании тестов: если механизм митигации реализован, у него должен быть тест, воспроизводящий соответствующий отказ без него.

12.3 Метрики валидации Proof of Concept

Критерии успешности PoC сведены в таблицу 9. Пороговые значения — начальные ориентиры для конфигурации `organism_default.yaml`; по мере накопления экспериментального опыта они подлежат калибровке, но сам набор метрик фиксирован: он покрывает все четыре группы свойств (биофизика, обучение, гомеостаз, производительность).

Таблица 8. Матрица ключевых рисков и встроенных механизмов митигации.

Риск	**Эффект**	**Митигация (раздел)**
Эпилептиформный спайковый шторм	лавинообразная синхронизация, потеря информации	адаптивные пороги ALIF (7) (разд. 3); латеральное GABA-торможение (9) (разд. 2.4)
Катастрофическое забывание при сборке	разрушение навыков Фазы 1 в Фазе 2	жёсткая заморозка w_{slow}; EWC (13); только положительная награда (14) (разд. 5)
Расходимость Reward-STDP	неограниченный рост w_{fast}, «съедание» базиса	амплитудный захват $\lvert w_{\text{fast}}\rvert \le w_{\max}$; медленное затухание буфера; обновление только при DA > 0 (разд. 5.2)
PCIe-узкое место	деградация скорости при обмене GPU $\leftrightarrow$ CPU	все модуляторы и токи — GPU-тензоры в общем графе; CPU — только мета-контур $K \approx 100$–500 шагов (разд. 8)
Вырождение полушарий	дублирование репрезентаций, потеря специализации	слом симметрии: разные τ_m/шум/5-HT/LR + Зона C не видит токены в Фазе 1 (разд. 2.2)
«Шизофренический» выход	одновременная трансляция несогласованных гипотез	Global Workspace: GABA-конкуренция WTA за выходной токен (разд. 2.4)
O(N^2)-взрыв памяти следов	невыполнимость 3-факторного правила на реальных сетях	векторная факторизация следов (11)–(12), память строго $O(N)$ (разд. 5.2)
BPTT-взрыв памяти Фазы 1	граф растёт с длиной последовательности	пошаговый `detach()` состояний, суррогатный градиент через текущие токи (разд. 7.1)

Таблица 9. Метрики валидации PoC по этапам дорожной карты.

Этап	Метрика	Пороговое значение / метод
0	NMDA-коинциденция	апикальный вход любой силы $\Rightarrow$ 0 спайков; совпадение слабого базального с апикальным $\Rightarrow$ спайки > 0 (тест из разд. 9.5)
0	Память следов	размер следов строго $[B, N_{\text{pre}}]$ и $[B, N_{\text{post}}]$; $\Delta w_{\text{slow}} = 0$ при активной Фазе 2
1	Изолированное обучение Зоны A	монотонное снижение $\mathcal{L}_{\text{A}}$ до плато ($\Delta\mathcal{L} < \epsilon$) без связей с Зоной C
1	Surprise-гейтинг SON	доля тактов активности SON $< 30\%$ на предсказуемом потоке
2	Слом симметрии	статистически значимое расхождение распределений спайков левого и правого полушарий на одном входе
2	WTA в Workspace	единственная доминирующая зона на каждом выходном такте; отсутствие штормов (частота спайков в норме)
3	Стойкость к забыванию	деградация $\mathcal{L}_{\text{A}}$ после Фазы 2 $\leq 5\%$ относительно снапшота
3	Сборка Reward-STDP	рост среднего вознаграждения на валидационных сессиях при положительном DA; отсутствие роста при DA $= 0$
4	Цикл сна	после консолидации качество задач Фазы 1 не ниже, шум w_{fast} (норма буфера) снижается
5	Инверсия правил	адаптивное переключение поведения при инверсии правил без переобучения базового синтаксиса (ключевой эксперимент)
5	Быстрый контур	профиль через CUDA Graphs; цель — снижение оверхеда такта $\geq 2\times$ относительно наивного запуска ядер

13 Глоссарий терминов и сокращений

Таблица 10. Глоссарий терминов и сокращений.

Термин	Значение
ALIF	Adaptive Leaky Integrate-and-Fire — LIF-нейрон с адаптивным порогом спайка (7).
AMPA	быстрые глутаматные ионотропные рецепторы; прямой возбуждающий ток Bottom-Up (3).
ACh	ацетилхолин — модулятор баланса Bottom-Up/Top-Down (5).
BPTT	Backpropagation Through Time — обратное распространение сквозь время; в Фазе 1 замещается SLTT ради памяти $O(1)$.
Corpus Callosum	«мозолистое тело» — асимметричная разреженная шина между полушариями с буфером задержки 5–10 шагов и GABA-фильтром.
DA	дофамин — ошибка предсказания награды (RPE), разрешающий гейт пластичности Фазы 2 (14).
Eligibility trace	след пригодности — затухающая метка синапса-кандидата на усиление; факторизована в $O(N)$ (11).

Термин	Значение
EWC	Elastic Weight Consolidation — регуляризация (13), подтягивающая веса к снапшоту Фазы 1 по матрице Фишера.
GABA	главный тормозной нейротрансмиттер; латеральное торможение (9), WTA-конкуренция, фильтрация шины.
Global Workspace	глобальное рабочее пространство — арена конкуренции зон за выходной токен (Soft-WTA).
LIF	Leaky Integrate-and-Fire — простейшая спайковая модель нейрона с утечкой.
LTD / LTP	долгосрочная депрессия / потенцирование — направления изменения синаптического веса в STDP (12).
Mg^{2+}-блок	потенциал-зависимая блокировка NMDA-каналов ионами магния; формализована гейтом (4).
NE	норадреналин — фазический аларм при Surprise; сброс адаптации, форсирование темпа обучения.
NMDA	медленные глутаматные рецепторы-детекторы совпадений; физический субстрат внимания и пластичности.
RPE	Reward Prediction Error — $R(t) - V_{\text{baseline}}$, сигнал дофаминовой системы.
Reset-by-subtraction	сброс мембраны вычитанием порога вместо обнуления; сохраняет градиентную информацию о превышении.
Reward-STDP	трёхфакторное правило пластичности «пре $\times$ пост $\times$ награда» (10).
SLTT	Spatio-Temporal Surrogate-Gradient Learning — обучение зон суррогатными градиентами с памятью $O(1)$ по времени.
SON	Self-Observation Neurons — ансамбль из 16 нейронов на зону: шина сжатия состояния, метакогниция, Surprise (17).
STDP	Spike-Timing-Dependent Plasticity — пластичность, зависящая от относительных моментов пре- и постсинаптических спайков.
Surprise	неожиданность — ошибка предсказания SON собственного следующего состояния (17).
Tag-and-Capture	механизм «мечения и захвата»: след пригодности живёт дольше задержки награды и захватывается дофамином.
WTA	Winner-Takes-All — конкуренция с выявлением единственного победителя; в мягкой форме — Soft-WTA.
w_{fast}	быстрая синаптическая память — буфер Фазы 2 с амплитудным ограничением и затуханием.
w_{slow}	долговременная синаптическая память — кристаллизованный базис навыков, обучается в Фазе 1 и консолидируется во сне (21).
5-HT	серотонин — регулятор энтропии/стабильности и триггер давления сна (20).

14 Заключение: следующий шаг — Этап 1

Фундамент Этапа 0 полностью спроектирован, реализован и верифицирован: ядро `core/` несёт компартментную биофизику, честную память следов $O(N)$ и контракты обеих фаз обучения. Следующий шаг по дорожной карте 6 — **Этап 1: изолированные функциональные зоны и модуляторный контур**, а именно три модуля: `neurochemistry.py` (динамика GPU-тензоров DA, 5-HT, ACh, NE), `son.py` (16 нейронов SON, авторегрессионное предсказание состояния зоны, вычисление Surprise, триггер норадреналина) и `zone.py` с изолированным тестом обучения Фазы 1 (`test_zone_isolated.py`). Порядок не случаен: модуляторный контур и шина SON — предпосылки для честной сборки зон, а не её последствия.

Дальнейшая перспектива проекта выходит за рамки одного Proof of Concept. Если три первых этапа докажут жизнеспособность локального обучения с дофаминовой сборкой, а Этапы 4–5 — воспроизводимый цикл сна и адаптивное переключение поведения при инверсии правил, то NEUROWEAVE получит то, чего сегодня нет ни у трансформеров, ни у SNN-фреймворков: непрерывно живущий субстрат, который учится в момент генерации, не забывает пройденное и знает, когда ему пора спать. Это и есть фундамент для ИИ нового поколения, ради которого проект затевался.